

Inheritance and Interfaces

Inheritance

The classes created in object-oriented programming (OOP) languages can inherit data and methods from existing classes.

Inheritance allows one (1) class to acquire all the behaviors and attributes of another class. When a class is created by inheriting from another class, the new class automatically contains the data fields and methods of the original class.

The extends keyword establishes inheritance between classes. A subclass extends a single-parent class and gains all its attributes and behaviors.

For example, a superclass called `Vehicle` defines behaviors like having wheels, an engine, and the ability to move. Then, the subclasses can inherit from `Vehicles` like cars, bikes, boats, etc.

```
public class Car extends Vehicle {  
    // car-specific fields and methods  
}
```

In the code snippet above, the `Car` subclass now inherits everything defined in the `Vehicle` class, such as wheels, engine, `move()` method, etc., and can add new fields and behaviors related to a car. Note that this does not replace the parent class, and both could still be used independently as needed.

Based on this, inheritance promotes **code reusability** by allowing the subclass to reuse fields and methods of the superclass. Code reuse can be done if, for instance, an existing class can be used and derived from a new class. Not only does it facilitate code reuse, but it also makes the code more modular and maintainable over time.

Here is another code example of performing inheritance.

```
class Planet {  
    // methods and fields  
}  
  
// use of extends keyword  
// to perform inheritance  
class Earth extends Planet {  
    // methods and fields of Planet  
    // methods and fields of Earth  
}
```

In the example, the `Earth` class (subclass) is created by inheriting the methods and fields from the `Planet` class (superclass).

Inheritance also provides an “**Is-A**” relationship between classes. If Class B inherits from Class A, it can be concluded that “B is a type of A”. Using the examples above, a car IS A vehicle, so it inherits from the `Vehicle` class. Earth IS A planet. Inheritance can only be used if there exists an is-a relationship between two (2) classes.

This relationship improves code clarity and reflects real-world relationships.

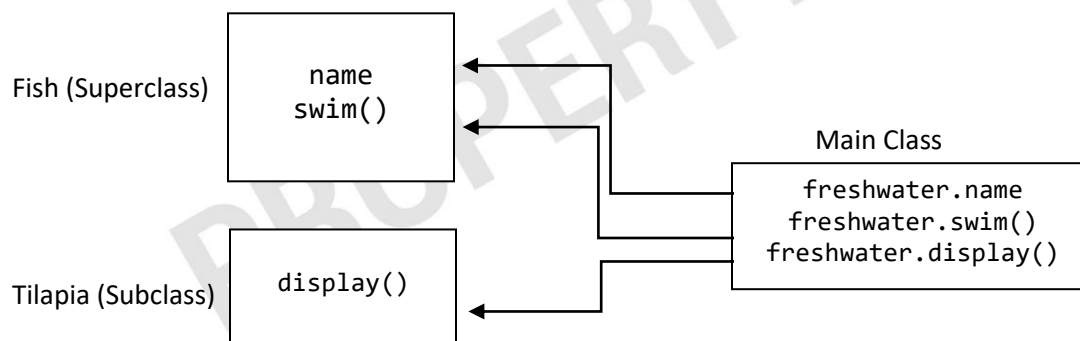
Example program:

```
class Fish {  
    // field and method of the parent class  
    String name;  
    public void swim() {  
        System.out.println("I can swim");  
    }  
}  
// Tilapia inherits from Fish  
class Tilapia extends Fish {  
  
    // new method in subclass  
    public void display() {  
        System.out.println("My name is " + name);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        // create an object of the subclass  
        Tilapia freshwater = new Tilapia();  
  
        // access field of superclass  
        freshwater.name = "Tippy";  
        freshwater.display();  
  
        // call method of superclass  
        // using object of subclass  
        freshwater.swim();  
    }  
}
```

Output:

```
My name is Tippy  
I can swim
```

In this example, a Tilapia subclass is derived from the Fish superclass. In particular, freshwater is an object of Tilapia, while name and swim() are members of the Fish class. Since the Tilapia subclass inherits the fields and methods from the Fish superclass, it was possible to access the fields and methods using the object of the Tilapia.



Method Overriding

This happens when the same method is present in both the superclass and subclass. It is a key feature of overriding or extending existing methods of the superclass. This is achieved by using the same name and signature for the method.

For example:

<pre>class Fish { // method in the superclass public void swim() { System.out.println("I can swim"); } } // Tilapia inherits from Fish class Tilapia extends Fish { // overriding the swim() method @Override public void swim() { System.out.println("I swim in freshwater"); } // new method in subclass public void eat(){ System.out.println("I can eat"); } } class Main { public static void main(String[] args) { // create an object of the subclass Tilapia freshwater = new Tilapia(); // call the swim() method freshwater.swim(); freshwater.eat(); } }</pre>	Output: <pre>I swim in freshwater I can eat</pre>
---	--

In this example, the swim() method is present in both the Fish superclass and the Tilapia subclass. Then, an object of freshwater of the Tilapia subclass is created. The method inside Tilapia can be called when the swim() method is called using the freshwater object.

super Keyword in Inheritance

Since the same method in the subclass overrides the method in the superclass, the super keyword can be used to call the method of the superclass from the method of the subclass.

For example:

<pre>class Fish { // method in the superclass public void swim() { System.out.println("I can swim"); } } // Tilapia inherits from Fish class Tilapia extends Fish { // overriding the swim() method @Override public void swim() { // call method of superclass super.swim(); System.out.println("I swim in freshwater"); } // new method in subclass public void eat(){ System.out.println("I can eat"); } } class Main { public static void main(String[] args) { // create an object of the subclass Tilapia freshwater = new Tilapia(); // call the swim() method freshwater.swim(); freshwater.eat(); } }</pre>	Output: <pre>I can swim I swim in freshwater I can eat</pre>
--	---

In this example, the swim() method is in both the Fish superclass and the Tilapia subclass. In the bolded part, the super keyword is used to call the swim() method in the superclass.

Interfaces

An **interface** allows the definition of a construct for a class without specifying its implementation. It creates a way to declare a set of functionalities that must be implemented. It is also a collection of abstract methods (methods without a body) and constant values.

The interface keyword is used to create an interface, such as:

```
interface Code {  
    public void getType();  
    public void getVersion();  
}
```

To implement an interface, it is important to note that objects of interfaces cannot be created. It must be implemented by other classes. The implements keyword is used to apply interface.

```
interface Shape {  
    void getVolume(int length, int width, int height);  
}  
// implement the Triangle interface  
class Box implements Shape {  
    // implementation of abstract method  
    public void getVolume(int length, int width, int height) {  
        System.out.println("The volume of a box is " + (length * width * height) + "  
meters." );  
    }  
}  
class Main {  
    public static void main(String[] args) {  
        Box b1 = new Box();  
        b1.getVolume(7,8,9);  
    }  
}
```

Output:
The volume of a box is 504 meters.

In this example, an interface named Shape is created, which contains an abstract method called getVolume(). The Box class implements the Shape interface, providing the implementation of the getVolume() method.

Another example that shows how an interface is implemented:

```
// create an interface  
interface Model {  
    void getType(String name);  
}  
  
// class implements interface  
class MachineLearning implements Model {  
  
    // implementation of abstract method  
    public void getType(String name) {  
        System.out.println("Machine Learning: " + name);  
    }  
}
```

```
}  
class Main {  
    public static void main(String[] args) {  
        MachineLearning model = new MachineLearning();  
        model.getType("Supervised Learning");  
    }  
}
```

Output:
Machine Learning: Supervised Learning

In this example, the Model interface is created, which includes the abstract method `getType()`. The MachineLearning class implements the interface and provides the implementation for the method.

It is also possible to implement **multiple interfaces**:

```
interface X {  
    // members of X  
}  
  
interface Y {  
    // members of Y  
}  
  
class Z implements X, Y {  
    // abstract members of X  
    // abstract members of Y  
}
```

Additionally, an interface can be extended using the **extends** keyword. For example:

```
interface Sports {  
    // members of Sports interface  
}  
  
// extending interface  
interface Basketball extends Sports {  
    // members of Basketball interface  
    // members of Sports interface  
}
```

In this example, the Basketball interface extends the Sports interface. If a class implements Basketball, it should provide implementations for all the abstract methods defined in both Sports and Basketball.

References:

- Programiz (n.d.a). *Java inheritance*. [Web Article]. Retrieved on January 6, 2025, from <https://www.programiz.com/java-programming/inheritance>
- Pulok, M. (2024). *Learn advanced Java: Programming with beginners*.
- Robbins, P. (2024). *Python, Java, SQL, & JavaScript: The ultimate crash course for beginners to master the 4 most in-demand programming languages, stand out from the crowd and find high-paying jobs*.